GUÍA COMPLETA · V0.1.0

# VOZ

## Dictado por voz para la terminal, local y sin nube. Arquitectura, uso y mantenimiento.

Para quien quiere entender cómo funciona por dentro, mantenerlo o hacer un fork. Documenta lo que el código hace hoy, con los datos medidos detrás de cada decisión.

**Autor**
Daniel Patiño Vargas
Colombia

**Versión**
0.1.0
29 de octubre de 2026

**Plataforma**
macOS, Apple Silicon
Python 3.12

**Licencia**
MIT

# Índice


**1. Qué es y filosofía** ..... **3**

**2. Instalación y requisitos** ..... **3**

**3. Uso completo** ..... **4**

3.1 Comandos ..... 4

3.2 Banderas ..... 5

3.3 Combinaciones que el parser rechaza ..... 6

3.4 Códigos de salida ..... 6

3.5 Interfaz y teclas del modo en vivo ..... 6

3.6 Tres formas de usar el micrófono ..... 7

3.7 Ejemplos ..... 8

**4. Arquitectura** ..... **8**

4.1 voz.py · CLI y orquestación ..... 8

4.2 record.py · captura y detección de voz ..... 9

4.3 live.py · tramos en vivo ..... 9

4.4 stt.py · transcripción y filtros ..... 10

4.5 corrections.py · glosario ..... 10

4.6 output.py · salida ..... 10

4.7 autosave.py · grabación a salvo ..... 11

4.8 liveui.py · el Shell de la interfaz ..... 11

4.9 flow.py · máquina de estados ..... 11

4.10 ui.py · terminal ..... 12

4.11 doctor.py y config.py ..... 12

**5. Flujo de datos de un dictado** ..... **12**

**6. Decisiones de diseño y por qué** ..... **13**

**7. Configuración** ..... **15**

7.1 Archivos y carpetas ..... 15

7.2 config.toml ..... 15

7.3 corrections.toml ..... 15

7.4 Variables de entorno ..... 16

7.5 Recuperar una sesión ..... 16

7.6 Recuperar el texto con voz copiar ..... 16

**8. Tests y benchmarks** ..... **16**

8.1 Tests ..... 16

8.2 Benchmarks ..... 17

8.3 Grabar tus 3 frases para bench_src.py ..... 18

**9. Guía para forks** ..... **18**

9.1 Enchufar otro motor de transcripción ..... 18

9.2 Portar a Windows o Linux ..... 19

9.3 Añadir un subcomando ..... 19

9.4 Cómo contribuir y versionar ..... 20

**10. Limitaciones conocidas y roadmap** ..... **20**

10.1 Limitaciones ..... 20

10.2 Roadmap ..... 20

**11. Licencia** ..... **21**

# 1. Qué es y filosofía

`voz` es una herramienta de línea de comandos que convierte tu voz en texto, en tu propio Mac, con mlx-whisper (Whisper de OpenAI ejecutado con MLX, el framework de Apple para sus chips). La entrada es el micrófono o un archivo de audio. La salida es la transcripción por *stdout*, más el portapapeles, y opcionalmente un archivo de texto, un `.srt` o un `.json`.

Nació para dictarle a un agente de código desde la terminal. El portapapeles es la integración: no hay plugin, solo `Cmd-V`. De ahí salió también la transcripción de notas de voz y de reuniones grabadas.

## Principios

- **Local y privado.** El audio y el texto no salen del equipo. Con el modelo en caché, el proceso no abre conexiones (ver sección 7). La única salida a la red es la descarga del modelo la primera vez.
- **La terminal primero.** En stdout va solo la transcripción; todo lo demás (estado, avisos, interfaz) va a stderr. Por eso `voz archivo.m4a | pbcopy` o una redirección funcionan sin ensuciar nada.
- **Nunca perder una grabación.** Mientras grabas, el audio y el texto confirmado se escriben a disco sobre la marcha. Descartar una grabación no borra el audio y se puede deshacer.
- **Pocas dependencias.** La interfaz de terminal usa solo secuencias ANSI y `termios`. No hay librerías de interfaz.
- **Medir antes de decidir.** Cada umbral y cada filtro de este proyecto sale de una medición (sección 6), y las decisiones cerradas se documentan con su dato para no reabrirlas por intuición.

## Lo que no es

No es una app con ventana, no separa voces (diarización), no captura el audio del sistema (por ejemplo, el de una videollamada) y no expone una API. Solo corre en macOS con chip Apple Silicon. La sección 10 detalla estas limitaciones y lo que sigue.

# 2. Instalación y requisitos

| Requisito | Detalle |
|---|---|
| Sistema | macOS con Apple Silicon (M1 o posterior): `uname -m` debe decir `arm64`. El motor MLX y `pbcopy` / `pbpaste` lo exigen. |
| Python | 3.12 ( `brew install python@3.12` si no lo tienes). `voz doctor` marca un aviso si usas otra versión. |
| Espacio | Entorno virtual de unos 1,1 GB (mlx-whisper arrastra torch). Modelo turbo, unos 1,6 GB. Modelo `--accurate`, unos 3 GB más la primera vez que lo uses. |
| ffmpeg | Solo para transcribir archivos comprimidos. El dictado por micrófono no lo usa. `brew install ffmpeg`. `voz doctor` lo marca como aviso, no como problema crítico. |
| Micrófono | Permiso de macOS para la aplicación de terminal que uses. |

### Pasos

```
uname -m                                  # arm64
python3.12 --version                      # si falta: brew install python@3.12
git clone https://github.com/danpv95/voz voz
cd voz
python3.12 -m venv .venv
./.venv/bin/python -m pip install -r requirements.lock.txt

mkdir -p ~/.local/bin
ln -sf "$PWD/bin/voz" ~/.local/bin/voz
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc   # si which voz no lo encuentra
source ~/.zshrc

voz doctor                                # verificación sin red
```

`requirements.txt` pide `mlx-whisper>=0.4.3`, `sounddevice>=0.5` (trae PortAudio dentro de su wheel de macOS, no hace falta `brew install portaudio`) y `numpy`. SciPy llega como dependencia de mlx-whisper y `voz` lo usa directamente para el filtro de banda. `requirements.lock.txt` fija las versiones exactas del entorno con el que se probó esta versión y es la forma recomendada de instalar: la instalación sale igual en cualquier equipo. `requirements.txt` solo pide mínimos y toma las últimas versiones; úsalo si prefieres actualizar.

> **No borres ni muevas la carpeta clonada: `voz` vive ahí.** El comando es un enlace a `bin/voz` y usa el `.venv` de esa carpeta. Si la mueves, repite el enlace.

### El wrapper `bin/voz`

Es un script `sh` de pocas líneas. Resuelve su propia ubicación siguiendo enlaces simbólicos (por eso funciona el `ln -sf` de arriba), fija las variables de entorno de la sección 7 y hace `exec` del Python del entorno virtual del repo con `src/voz.py`. No hay ruta fija en ninguna parte.

### Permiso de micrófono

macOS concede el permiso a la aplicación de terminal, no al script. Si `voz` muestra «✖ No llega audio del micrófono» (en el flujo simple, «Silencio detectado»), activa tu terminal en Ajustes del Sistema, Privacidad y seguridad, Micrófono, y pulsa `↵` para reintentar: la interfaz vuelve a consultar el dispositivo por defecto. Nunca muestra un traceback. El aviso del flujo simple también sugiere `tccutil reset Microphone` con el identificador de Terminal o de iTerm, que reinicia el permiso de esa aplicación y de ninguna otra.

> **Primera ejecución.** La primera transcripción descarga el modelo a `~/.cache/huggingface/`. `voz` lo muestra en la propia interfaz (`Descargando modelo · 420 MB / ~1,6 GB · esc salir`, y después `Cargando modelo… N s`; sin cifras fiables, `Descargando modelo (~1,6 GB, solo la primera vez)… N s`) y vuelve a poner el modo sin conexión después. Una descarga cortada continúa en el siguiente arranque.

# 3. Uso completo

## 3.1 Comandos

| Comando | Qué hace |
|---|---|
| `voz` | Dicta por el micrófono con transcripción en vivo y copia al terminar. |
| `voz <archivo>` | Transcribe un archivo de audio (m4a, ogg, opus, mp3, wav; lo que ffmpeg decodifique). |
| `voz doctor` | Revisa Python, módulos, modelos (disponible, incompleto si faltan los pesos, o sin descargar), ffmpeg (aviso si falta), micrófono por defecto, glosario y el comando `voz`. Sin red. Con `--mic` graba 1 s y mide el nivel. |

| Comando | Qué hace |
|---|---|
| `voz copiar [N]` | Vuelve a copiar al portapapeles una grabación guardada en la carpeta de estado (la última no descartada; `N` cuenta de la más nueva a la más vieja). `--lista` las muestra numeradas, `--todas` une las de la última sesión y `--stdout` imprime en vez de copiar. Solo lee: no escribe nada. |
| `voz devices` | Imprime la lista de dispositivos de audio que ve PortAudio ( `sounddevice.query_devices()` ). Sirve para depurar. |
| `voz --version` | Imprime `voz 0.1.0` y sale. |

## 3.2 Banderas

Estas son todas las del parser ( `build_parser` en `src/voz.py` ). `--help` los muestra en español junto con las teclas del modo en vivo y las rutas de configuración. Existe además `--sim` , interna y oculta de `--help` , que alimenta la interfaz con un archivo de audio en lugar del micrófono para pruebas y demos.

| Bandera | Aplica a | Default | Efecto |
|---|---|---|---|
| `--loop` | micrófono | apagado | Repite sin salir. Con la interfaz interactiva no cambia nada (grabar otra, `r` , ya está incorporado). En el flujo simple o con `--no-live` repite dictados sin recargar el modelo; `Ctrl-C` sale. |
| `--model REPO` | todo | turbo | Repositorio de mlx-whisper. Default `mlx-community/whisper-large-v3-turbo` . Excluyente con `--accurate` . |
| `--accurate` | todo | apagado | Usa `mlx-community/whisper-large-v3-mlx` : más lento y más preciso. |
| `--lang L` | todo | `auto` | `auto` mantiene el idioma de la sesión: empieza por tu idioma principal ( `[idioma] principal` , si no el del sistema, si no español) y solo cambia con evidencia fuerte; en archivos detecta por frase. Mezclar es/en en una sesión sigue funcionando cuando el cambio es claro; en llamadas o con varias voces conviene `--lang es` . Otro valor ( `es` , `en` , ...) lo fija. El parser no lo valida todavía: un código inválido falla más tarde. |
| `--out RUTA` | todo | ninguno | Además, añade el texto al final de ese archivo (con salto de línea). Si no se puede escribir (sin permiso, carpeta inexistente), la interfaz y el flujo simple avisan con un mensaje claro y salen con código 1; el texto sigue en el portapapeles y en la carpeta de estado. |
| `--no-copy` | todo | apagado | No toca el portapapeles. Gana sobre la configuración. |
| `--no-live` | micrófono | apagado | Modo anterior: graba todo ( ↵ empieza, ↵ termina) y transcribe al final. |
| `-y` , `--yes` | micrófono | apagado | En el flujo simple en vivo, copia sin preguntar. La interfaz interactiva se rige por la configuración `copiar` . |
| `--no-trim` | micrófono con `--no-live` | apagado | No recorta el silencio del inicio y del final de la grabación. |
| `--srt` | archivo | apagado | Escribe `<audio>.srt` con tiempos, junto al audio. |
| `--json` | archivo | apagado | Escribe `<audio>.json` con `start` , `end` , `text` y, en `auto` , `lang` . |
| `--mic` | `doctor` | apagado | Graba 1 s y mide el micrófono. |
| `--version` | - | - | Imprime la versión. |

## 3.3 Combinaciones que el parser rechaza

Terminan con el código de salida 2 y un mensaje de uso.

- `--srt` o `--json` sin un archivo de audio.
- `--loop` junto a un archivo o a `doctor` / `devices`.
- `-y` junto a `--no-copy` (se contradicen).
- `--no-trim` con el micrófono sin `--no-live` (el modo en vivo ya corta por pausas).
- `--model` junto a `--accurate`.

## 3.4 Códigos de salida

| Código | Significa |
|---|---|
| 0 | Todo bien. También al salir con `esc` desde «Listo» sin haber grabado. |
| 1 | No se transcribió nada, no hay voz, el archivo no existe, falló la carga del modelo o no se pudo escribir en `--out` (en la interfaz y el flujo simple). |
| 2 | Error de uso de banderas, o `--sim` fuera de la interfaz interactiva. |
| 130 | Interrumpido con `Ctrl-C`, siempre, haya texto o no. El texto se conserva en el portapapeles (según la configuración), en stdout si no es una terminal y en `--out`. |

## 3.5 Interfaz y teclas del modo en vivo

Así se ve una sesión. Los tiempos y el texto cambian con lo que digas.

```
voz  ·  turbo  ·  auto  ·  🎙 MacBook Pro Microphone
⁖ Cargando modelo… 3 s   esc salir

● Listo   ↵ hablar · esc salir
  habla con pausas cortas: voz transcribe mientras hablas · voz --help para más

● Escuchando  00:12  ·····▃▅··  es   ↵ terminar · p pausa · esc descartar
⁖ Terminando… 2 s · 1 tramo pendiente
```

| Elemento | Qué significa |
|---|---|
| Cabecera | Modelo abreviado (`turbo`, o `large-v3` con `--accurate`), idioma y, tras el icono 🎙, el micrófono de entrada por defecto del sistema. Si no es el que esperas, cámbialo en los ajustes de sonido de macOS. |
| Descargando / cargando modelo | La primera vez, `⁖ Descargando modelo · 420 MB / ~1,6 GB · esc salir`; después (y siempre en los arranques siguientes) `⁖ Cargando modelo… 3 s` con un contador y la sugerencia `esc salir`. Un `↵` en este momento se ignora con el aviso «↵ ignorado: el modelo todavía está cargando». |
| Pista de primer uso | En gris bajo «Listo», solo la primera vez: habla con pausas cortas, porque `voz` transcribe mientras hablas. |
| Medidor | En silencio, puntos tenues (`·`). Con voz detectada, barras de color (`▃▅▇`). Las barras de color significan que `voz` te está detectando; si hablas y solo ves puntos, revisa el micrófono (`voz doctor --mic`). |
| Terminal estrecha | Si la línea de estado no cabe (menos de unas 60 columnas), las teclas pasan a una segunda línea propia; se recorta antes el medidor y los detalles, nunca las teclas. Los avisos de pérdida de audio van en su propia línea. |
| Terminando | `⁖ Terminando… 2 s · 1 tramo pendiente`: un contador de segundos y cuántos tramos faltan por transcribir antes de la revisión. Espera como máximo 20 s: si el transcriptor no responde, o pulsas `Ctrl-C` otra vez, sale igual; el WAV queda guardado, la sesión se marca incompleta (el `.lock` se conserva) y el siguiente `voz` ofrece re-transcribirla. |

Las teclas son las de la máquina de estados `flow.py`. Las letras no distinguen mayúsculas.

| Estado | Teclas |
|---|---|
| Listo | `↵` o espacio: hablar · `esc`: salir. Mientras el modelo carga (`⁘ Cargando modelo… N s`), un `↵` se ignora con un aviso visible; si la carga falla, se muestra el error y su causa sin pulsar nada. |
| Escuchando | `↵` o espacio: terminar · `p`: pausa · `esc`: terminar y descartar (deshacible) |
| En pausa | `p`: reanudar · `↵`: terminar · `esc`: terminar y descartar. El micrófono sigue abierto pero no se graba ni se transcribe. |
| Revisión | `↵` o `esc`: salir (el texto ya está copiado: `Cmd-V` para pegar; la pista solo lo dice si de verdad se copió) · `r`: grabar otra · `c`: añadir a esta grabación · `⌫`: descartar · `u`: deshacer el descarte · `s`: copiar (solo si la configuración no es `auto` ni se usó `--no-copy`) |
| Sin voz | `r`: intentar de nuevo · `↵` o `esc`: salir |
| Sin audio del micrófono | El micrófono no se pudo abrir o solo entregó silencio digital: «✖ No llega audio del micrófono», la ruta de Ajustes y `↵` (o `r`): reintentar · `esc`: salir. |
| Texto incompleto | Si algún tramo no se transcribió, la revisión muestra «⚠ Texto incompleto · N tramos sin transcribir · copiado» y el comando para transcribir el audio entero. Si fallaron todos, en lugar de «No se detectó voz»: «✖ No se pudo transcribir · el audio quedó guardado». |
| Elegir (salir con 2 o más grabaciones válidas) | `↵`: dejar copiada la última · `a`: copiar todas juntas · `esc`: salir sin cambiar el portapapeles |

- El espacio equivale a `↵` solo en «Listo» y «Escuchando».
- `Ctrl-C` es una salida segura en cualquier estado: termina la grabación en curso, la guarda y deja copiado el último texto válido (según `copiar`). Sale siempre con código 130, haya texto o no, y dice lo que pasó: «Interrumpido · texto copiado», «Interrumpido · audio guardado en *ruta*» o «Cancelado» (no había grabación); nunca promete un respaldo que falló.
- `Ctrl-\` vuelca la pila de todos los hilos a stderr. Es la herramienta para diagnosticar un congelamiento.
- Pegar texto en la terminal (bracketed paste) se traga entero y no dispara teclas. Las flechas se ignoran.

## 3.6 Tres formas de usar el micrófono

| Modo | Cuándo se usa | Cómo funciona |
|---|---|---|
| Interfaz interactiva (`liveui`) | Por defecto, si stdin y stderr son una terminal y no pusiste `VOZ_UI=simple`. | Región redibujable en stderr, medidor de nivel, texto confirmado y provisional, teclas de la sección 3.5. |
| Flujo simple en vivo (`live.record_live`) | Sin terminal interactiva (pipes, scripts) o con `VOZ_UI=simple`. | Pide `↵` para empezar y para terminar. El texto va saliendo por stdout. Al final pregunta «¿Copiar al portapapeles? [S/n]»; sin terminal copia sin preguntar. `-y` evita la pregunta. |
| `--no-live` (`record.record_push_to_talk`) | Si lo pides. | Graba todo, recorta el silencio, transcribe al final y emite una sola vez. |

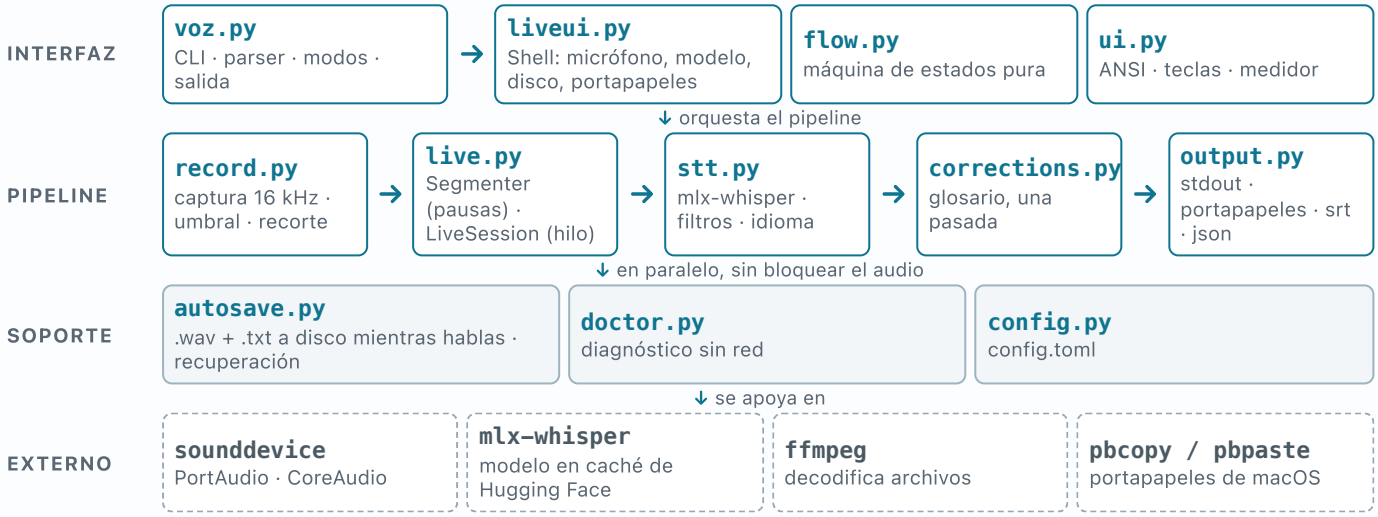
Si stdout no es una terminal (por ejemplo `voz > nota.txt`), la interfaz interactiva escribe en él, al salir, el texto elegido. En stdout nunca va nada más.

### 3.7 Ejemplos

```
voz                                 # dictar, ↵ empieza y ↵ termina
voz --lang es --accurate            # español fijo, modelo grande
voz --out ~/notas/hoy.md            # copia y además añade al archivo
voz nota.m4a                        # transcribir un audio
voz reunion.m4a --srt --json        # y escribir reunion.srt y reunion.json
voz reunion.m4a --no-copy > r.txt   # solo stdout, a un archivo
VOZ_UI=simple voz -y                # flujo simple, copia sin preguntar
voz doctor --mic                    # diagnóstico, con prueba de micrófono
voz copiar --lista                  # grabaciones guardadas, numeradas
voz copiar 2                        # recupera la penúltima al portapapeles
```

# 4. Arquitectura

El código vive en `src/` como módulos planos. `bin/voz` lanza `src/voz.py`, que añade su carpeta al `sys.path`, por eso los módulos se importan entre sí por nombre (`import stt`). Los imports pesados (MLX, sounddevice, SciPy) son perezosos para que `--help` y `voz devices` arranquen al instante.




Línea continua: módulos de `voz`. Discontinua: dependencias externas. Los archivos de audio no pasan por `live`: `voz.py` llama directo a `stt`, que corta en frases con `record.split_on_silence`.

### 4.1 `voz.py` · CLI y orquestación

RESPONSABILIDAD

Punto de entrada. Define el parser, valida combinaciones, decide qué modo ejecutar y se encarga del cierre del proceso.

FUNCIONES CLAVE

- `build_parser`, `parse_args`: banderas y validaciones; resuelve el modelo (`--accurate` contra `--model`).
- `main`: reparte entre `devices`, `doctor`, archivo, `--loop` y micrófono. Carga el glosario y, en modos de micrófono, avisa de sesiones sin cerrar (`autosave.report_unfinished`).
- `cmd_mic`, `cmd_loop`, `cmd_file`: los tres caminos. `_live_ui` y `_live_once` son la interfaz interactiva y el flujo simple.
- `_interactive`: interfaz solo si stdin y stderr son terminal y `VOZ_UI` no es `simple`.
- Al final, si `record.hard_exit_reasons()` no está vacío (el driver de audio no soltó el micrófono, el modelo seguía cargando), sale con `os._exit` tras avisar. El texto ya está escrito y copiado.

ARCHIVO

`src/voz.py`

### 4.2 `record.py` · captura y detección de voz

RESPONSABILIDAD

Abrir y cerrar el micrófono, y decidir qué es voz y qué es silencio.

FUNCIONES CLAVE

- `open_input` : `sounddevice.InputStream` mono, float32, a 16 kHz ( `SAMPLE_RATE` ), en el dispositivo de entrada por defecto del sistema. Cualquier fallo sube como `MicError` ; con `refresh=True` (el reintento) PortAudio vuelve a listar los dispositivos.
- `stop_stream` : detiene el micrófono en un hilo con tope de 1,5 s. En macOS, parar un micrófono real puede bloquearse en CoreAudio (sección 6); si no suelta, se marca y `voz.py` sale con `os._exit` .
- `adaptive_threshold(levels, profile)` : `max(min_abs, min(p10 * floor_mult, p99 * peak_frac))` . Dos perfiles: `LIVE_PROFILE` (3e-3, 1,5, 0,08) y `TRIM_PROFILE` (1e-3, 3,0, 0,02).
- `voice_band_sos` : filtro Butterworth de 4º orden, banda de 100 a 4000 Hz, para medir energía.
- `trim_silence` : quita silencio de los extremos con ventanas de 20 ms y 300 ms de margen. `split_on_silence` : corta archivos en frases. `rms` .
- `record_push_to_talk` , `prompt` (escribe el prompt en stderr).

ARCHIVO

`src/record.py`

### 4.3 `live.py` · tramos en vivo

RESPONSABILIDAD

Cortar el flujo del micrófono en tramos de habla y transcribirlos mientras sigues hablando.

FUNCIONES CLAVE

- `Segmenter.feed / current / flush` : recibe bloques de cualquier tamaño y devuelve tramos terminados. Mide energía por cuadros de 30 ms sobre la banda de voz con un umbral adaptativo sobre los últimos 5 s. Cierra un tramo tras 700 ms de silencio (mínimo 0,6 s), o a los 12 s con corte suave en el cuadro más silencioso del último 1,5 s. Conserva 300 ms previos al primer cuadro con voz. Los tramos que emite son el audio original, no el filtrado.
- `LiveSession` : un único hilo trabajador es dueño del modelo. Carga el modelo (calentamiento) mientras el usuario pulsa la primera tecla, transcribe los tramos que llegan por una cola y, entre tramos, hace transcripciones parciales del tramo en curso ( `PARTIAL_MIN_S` 0,8 s; espera audio nuevo equivalente a 1,2 veces lo que costó la anterior). Expone `submit` , `hold` / `release` , `drain` (esperar sin cerrar el hilo), `finish` , `committed` , `partial` , `failed_chunks` , `error` , `ready` .
- Puerta de silencio: un tramo cuyo pico (el mayor RMS de cuadro de 30 ms) queda bajo `MIN_CHUNK_PEAK` (0,015) no se manda a Whisper, que ante ruido o silencio inventa texto (a veces en inglés). Los parciales usan la misma puerta. Con `VOZ_DIAG=1` , cada tramo deja una línea (descartado, aceptado, segundos, pico, idioma, nº de caracteres; nunca el texto) en `diag.log` dentro de la carpeta de estado.
- `record_live` y `decide_copy` : el flujo simple.

ARCHIVO

`src/live.py`

### 4.4 `stt.py` · transcripción y filtros

RESPONSABILIDAD

Envolver mlx-whisper, elegir el idioma y limpiar lo que Whisper alucina.

FUNCIONES CLAVE

- `transcribe_segments(audio, *, model, language, initial_prompt, condition_on_previous_text, dictation, lang_state)` devuelve `[{start, end, text}]`. `audio` es una ruta o un arreglo float32 mono a 16 kHz. Con `dictation=True` endurece el umbral de no-voz (0,45) y recorta la escalera de temperaturas a (0,0, 0,2). Si falta el modelo en caché, o está incompleto, lo descarga una vez con `download_model`.
- `model_status(model)`: «ready» (configuración y pesos en la caché), «incomplete» (descarga interrumpida) o «missing», sin red. `download_model` baja el snapshot con `huggingface_hub.snapshot_download` y cuenta los bytes en `download` (`DownloadProgress`), que lee la interfaz; `format_size` da «420 MB», «1,6 GB».
- `transcribe`: lo mismo, unido en un texto.
- `_filter_segments`: descarta segmentos de baja confianza y sin voz, frases basura conocidas (`JUNK_PHRASES`: «gracias», «thank you», «amara.org»...) cuando el segmento entero es una de ellas y parece silencio o no-voz, y bucles de repetición.
- `_collapse_repeats`, `_collapse_word_runs`: colapsan la misma frase repetida 3 o más veces seguidas, y n-gramas de hasta 10 palabras (una palabra sola, 4 veces). Los dígitos y las letras sueltas se dictan literalmente y no se tocan.
- `detect_language_probs`: probabilidades del detector para español e inglés (`AUTO_LANGUAGES`). `LangState`: el idioma pegajoso de una sesión en vivo; `choose` es pura y `current` solo lo fijan los tramos confirmados con texto (ni los parciales, ni un error, ni un tramo filtrado a vacío lo mueven). `default_primary` resuelve el idioma principal una vez por proceso (`config.primary_language`). `detect_languages` (en lotes de 8): detecta por frase en archivos.
- `transcribe_file_segments`: para archivos en `auto`. Corta en frases (pausas desde 0,7 s), detecta el idioma de cada una, reúne frases contiguas del mismo idioma (hasta 25 s), transcribe y reajusta los tiempos a la línea de tiempo del archivo.
- `last_language`: idioma de la última transcripción, para la interfaz.

ARCHIVO

`src/stt.py`

### 4.5 `corrections.py` · glosario

RESPONSABILIDAD

Corregir nombres propios y términos técnicos que Whisper oye mal, sin red y sin tocar el modelo.

FUNCIONES CLAVE

- `load(path)`: lee `[replacements]` de `corrections.toml`. Ordena las claves de la más larga a la más corta, ignora claves de menos de 3 caracteres y reglas que solo cambian mayúsculas. Un archivo ilegible avisa y sigue sin correcciones: nunca rompe una transcripción. Cada clave se compila con límites de palabra Unicode, sin distinguir mayúsculas.
- `apply(text, rules)`: una sola pasada con un patrón combinado, de modo que la salida de una regla nunca dispara otra. El valor se escribe literal.

ARCHIVO

`src/corrections.py` · ejemplo en `config/corrections.example.toml`

### 4.6 `output.py` · salida

RESPONSABILIDAD

Entregar el texto: stdout, portapapeles, archivo y subtítulos.

FUNCIONES CLAVE

- `emit`: imprime el texto en stdout (lo único que va ahí), copia y añade a `--out`. Los fallos de copia o de escritura se avisan por stderr sin perder el texto.
- `to_clipboard`, `from_clipboard`: `pbcopy` y `pbpaste` en UTF-8.
- `to_srt`, `to_json`, `write_sidecar`, `unique_path`: escriben junto al audio y nunca sobrescriben (`-1`, `-2`...).

ARCHIVO

`src/output.py`

### 4.7 `autosave.py` · grabación a salvo

RESPONSABILIDAD

Que un congelamiento, un `kill` o un corte de luz no pierdan lo dicho.

FUNCIONES CLAVE

- `Session`: por cada grabación escribe `sesion-AAAAMMDD-HHMMSS-gN.wav` (16 kHz, mono, PCM de 16 bits), el `.txt` con una línea por tramo confirmado y un `.lock` con un `flock` mientras esté abierta. El nombre se reserva de forma atómica (el `.lock` y el `.wav` se crean con `O_EXCL`): si dos `voz` arrancan a la vez, o el bloqueo falla, la sesión toma el siguiente sufijo (`-2`, `-3` …) en vez de escribir sobre archivos ajenos. La carpeta queda en `0700` y los archivos en `0600`, sin importar el `umask`. El callback de audio solo encola; un hilo escritor hace el I/O para que el disco nunca haga perder muestras. Los errores de audio (`audio_failed`) y de texto (`text_failed`) van por separado: un fallo del `.txt` no detiene el respaldo del WAV, ni al revés, y el aviso se conserva hasta el cierre. El `.lock` se borra solo si el hilo escritor terminó bien y nada falló; si no, queda para que el siguiente arranque ofrezca la sesión. Si no se pueden restringir los permisos (`chmod`), avisa una vez. Reescribe el encabezado WAV cada segundo, así un cierre brusco deja un `.wav` válido.
- `new_run_id`: identificador único por ejecución, incluso si dos arrancan en el mismo segundo.
- `prune`: conserva las últimas 5 ejecuciones completas y nunca toca la actual ni una con grabación abierta.
- `unfinished`, `report_unfinished`: en el siguiente arranque detectan `.lock` huérfanos (el SO suelta el `flock` al morir el proceso) e imprimen la ruta y el comando para re-transcribir.

ARCHIVO

`src/autosave.py`

### 4.8 `liveui.py` · el Shell de la interfaz

RESPONSABILIDAD

Ejecutar lo que `flow.py` decide: abrir el micrófono, manejar el modelo, escribir a disco, copiar y dibujar.

FUNCIONES CLAVE

- `run` → `Shell.run`: bucle de dibujo y lectura de teclas cada 80 ms.
- `Audio`: el micrófono se abre una sola vez (con la primera pulsación) y se cierra al salir. Una compuerta (`gate`) descarta el audio mientras estás en pausa o en revisión. Cada grabación tiene su propio trío `Segmenter`, `LiveSession` y `autosave.Session`.
- `Shell.perform`: traduce cada acción (`START`, `PAUSE`, `FINISH`, `DISCARD`, `UNDO`, `COPY`, `EXIT`) en efectos. Al descartar, devuelve al portapapeles la grabación válida anterior o lo que tenías antes de abrir `voz`.
- `summary_line`, `lines`: textos de cada estado, con versión compacta por debajo de 80 columnas. `_status` pasa las teclas a una segunda línea si no caben; `_hint` reparte las teclas en las líneas necesarias sin cortarlas.
- `_open_mic`: si el micrófono no abre, la grabación termina en la pantalla «No llega audio del micrófono» (`flow.mic_failed`); el silencio digital total llega al mismo sitio.

ARCHIVO

`src/liveui.py`

### 4.9 `flow.py` · máquina de estados

RESPONSABILIDAD

Toda la lógica de teclas, sin tocar micrófono, modelo ni terminal. Por eso cada transición tiene prueba unitaria.

FUNCIONES CLAVE

- `step(flow, key)` devuelve la acción (`Act`) que el Shell debe ejecutar; las teclas desconocidas son `NOOP`. Estados: `READY`, `LISTEN`, `PAUSED`, `FINISHING`, `REVIEW`, `NOVOICE`, `CHOOSE`, `DONE`.
- `norm` (tecla cruda a tecla lógica), `finished` (el Shell informa el resultado de una grabación), `mic_failed` (el micrófono no abrió: la grabación queda sin audio y `↵` reintenta), `joined` (une las válidas con una línea en blanco).

ARCHIVO

`src/flow.py`

### 4.10 `ui.py` · terminal

RESPONSABILIDAD

Dibujar y leer teclas con la biblioteca estándar. Respeta `NO_COLOR`.

FUNCIONES CLAVE

- `Region`: bloque de líneas en stderr que se redibuja en su sitio; si la ventana cambia de tamaño, abre una región nueva en lugar de corromper la pantalla.
- `Keys`: lector en modo `cbreak` que devuelve una tecla lógica por llamada; traga el pegado entre corchetes, descarta flechas y separa un `esc` suelto de una secuencia.
- `meter` (barras sobre una escala de -60 a -10 dBFS; el medidor de la interfaz muestra puntos tenues en silencio y barras de color cuando detecta voz), `wrap_transcript` (confirmado en el color por defecto de la terminal, provisional en tono tenue (atributo «dim»), legibles con fondo claro y oscuro; solo ajusta la cola), `fit` (recorta a las columnas visibles), `status_line`.

ARCHIVO

`src/ui.py`

### 4.11 `doctor.py` y `config.py`

RESPONSABILIDAD

`doctor`: diagnóstico sin red. `config`: preferencias del usuario.

FUNCIONES CLAVE

- `doctor.run(mic)`: comprueba versión de Python, imports de `mlx_whisper`, `sounddevice` y `numpy`, el estado de los dos modelos (`stt.model_status`: disponible, incompleto si faltan los archivos de pesos, o ausente), `ffmpeg`, el micrófono por defecto, el número de reglas del glosario y si `voz` está en el PATH. Los problemas críticos (imports, micrófono) devuelven código 1; la falta de ffmpeg, un modelo ausente o incompleto y el resto son advertencias (`⚠`). Símbolos unificados: `✓` éxito, `✖` error, `⚠` advertencia, `●` estado, `·` separador. `_check_mic` graba 1 s y mide el RMS.
- `config.load()`: lee `config.toml`; un archivo ilegible (TOML roto o que no es UTF-8), un tipo incorrecto (por ejemplo `portapapeles = 1`) o un valor desconocido caen al default y avisan por stderr, sin traceback. Existen `[portapapeles] copiar` e `[idioma] principal` (`config.primary_language` resuelve el principal: config; `LC_ALL` / `LANG` si dicen `es_*`; la región de `AppleLocale`; español).

ARCHIVOS

`src/doctor.py`, `src/config.py`

## 5. Flujo de datos de un dictado

Recorrido de una sesión interactiva típica, desde que escribes `voz` hasta el `Cmd-V`.

1. **Arranque.** `bin/voz` fija el entorno y lanza `voz.py`. `main` carga el glosario, avisa de sesiones sin cerrar y entra a `_live_ui`.
2. **Calentamiento.** `Shell` crea la primera `LiveSession`. Su hilo carga el modelo transcribiendo 0,5 s de silencio (en español si el idioma es `auto`, porque detectar sobre silencio alucinaba «Thank you.») mientras la pantalla muestra «Cargando modelo…» con su contador y luego «Listo».
3. **Empezar.** Pulsas `↵`. `flow.step` devuelve `START`; el Shell espera a que el modelo esté listo, abre el micrófono (una vez) y activa la compuerta.
4. **Captura.** PortAudio entrega bloques float32 a 16 kHz. `Audio.feed` escribe el bloque a disco (`autosave`, solo encola) y se lo pasa al `Segmenter`.
5. **Corte por pausas.** El `Segmenter` mide la energía en la banda de voz y cierra un tramo tras 700 ms de silencio o a los 12 s. Cada tramo entra a la cola de la `LiveSession`.
6. **Transcripción.** El hilo trabajador toma el tramo y llama a `stt.transcribe`: idioma pegajoso (`LangState`) si es `auto`, mlx-whisper, y `_filter_segments` contra alucinaciones.
7. **Glosario y confirmación.** `corrections.apply` corrige el texto; el tramo pasa a `parts` (confirmado) y se anota en el `.txt` de la sesión.

8. **Texto provisional.** Entre tramos, el hilo transcribe lo que lleva el tramo en curso (`Segmenter.current`) y lo muestra en gris. Se reemplaza cuando llega la versión confirmada.
9. **Terminar.** Pulsas `↵`. El Shell cierra la compuerta, envía lo que quede (`Segmenter.flush`), espera a que se transcriba todo (`drain`, con un plazo de 20 s) y el estado pasa a «Terminando…» con un contador de segundos y los tramos pendientes.
10. **Entrega.** `flow.finished` pasa a «Revisión». El texto se imprime en el historial de la terminal (stderr), se copia con `pbcopy` (si el modo es `auto`) y se muestra el resumen: duración, palabras e idioma.
11. **Salida.** Con `↵` el micrófono se cierra con `stop_stream`, el autosave se cierra limpio (quita el `.lock`) y se podan las sesiones antiguas. Si stdout no es una terminal, el texto elegido se escribe ahí.

### Transcribir un archivo

Es un camino más corto, sin `record` ni `live`. Con `--lang` fijo, `voz.py` llama a `transcribe_segments(ruta, dictation=False)`; mlx-whisper decodifica con ffmpeg. Con `auto`, `transcribe_file_segments` corta en frases, detecta el idioma de cada una por lotes y transcribe grupos contiguos del mismo idioma. En ambos casos: glosario por segmento, texto a stdout, portapapeles y `--out` primero, y solo después el `.srt` y el `.json`. Así, una carpeta de solo lectura o una ruta mala cuestan el archivo auxiliar y nunca la transcripción.

## 6. Decisiones de diseño y por qué

Estas decisiones están cerradas. Cada una nació de una medición o de un fallo reproducido; si quieres reabrir una, trae un dato nuevo. Las medidas son del autor, con voz sintética del comando `say` de macOS y en un solo equipo: úsalas como orden de magnitud, no como garantía.

| Decisión | Valor | Por qué (dato) |
|---|---|---|
| Modelo por defecto | `whisper-large-v3-turbo` | Unas 5 veces más rápido que large-v3, con un WER unos 0,4 puntos porcentuales peor. Para reuniones importantes está `--accurate`. |
| `initial_prompt` | Siempre apagado (`None`) | Un prompt con vocabulario se repite textualmente sobre silencio o clips cortos (eco del prompt). Encadenar contexto entre tramos también empeoró los resultados. Los nombres propios se arreglan con el glosario. |
| Frecuencia de muestreo | 16 kHz mono | Whisper trabaja a 16 kHz y CoreAudio remuestrea de forma transparente. Grabar a 48 kHz solo triplicaría los datos. |
| Idioma | `auto` restringido a es/en | Sin restringir, los tramos cortos se detectaban como portugués o gallego. |
| Idioma en vivo | Pegajoso, con sesgo al idioma principal | El detector por tramo se voltea al inglés con audio lejano o de varias voces y Whisper inventa frases. Se cambia de idioma con 0,80 de probabilidad o 0,50 de ventaja; volver al principal pide 0,60 o 0,25 (se corrige un cambio falso en un tramo). El primer tramo siembra el principal salvo que el otro idioma sea claro, y los parciales antes del primer tramo usan el principal. |
| Idioma en archivos | Detección por frase, en lotes de 8, uniendo frases contiguas del mismo idioma (hasta 25 s) | Whisper detecta una sola vez con los primeros 30 s: en un archivo mixto salía en inglés 1 de 4 frases; con la detección por frase, 4 de 4. |
| Calentamiento | Idioma fijo `es` | En `auto`, sobre silencio, alucinaba «Thank you.» |
| Tramo máximo en vivo | 12 s con corte suave | Con 25 s cada parcial costaba hasta unos 1,2 s y el hilo trabajador no paraba de re-transcribir. |

| Decisión | Valor | Por qué (dato) |
|---|---|---|
| `word_timestamps`, `fp16=False` | No se usan | Añadían 145 % y 95 % de latencia sin mejorar el WER. |
| Normalizar números | No | Whisper ya convierte «tres» en 3 y «cuarenta y cinco mil» en 45.000. |
| Medir energía | Banda de 100 a 4000 Hz (IIR con estado); Whisper recibe el audio original | Subió los aciertos del detector con 20 dB de relación señal-ruido de 0 de 9 a 6 de 9. |
| Umbral de voz | Dos perfiles: en vivo (3e-3, 1,5, 0,08) y recorte (1e-3, 3,0, 0,02) | En vivo, un umbral bajo une todo en un tramo y corta a mitad de frase: debe resistir el ruido. En el recorte, equivocarse deja el audio sin recortar, que es el fallo seguro y protege las voces suaves. |
| «gracias» aislado | Se descarta solo si el audio en memoria es silencio (RMS menor que 0,01) | Un «gracias» real y uno alucinado tienen la misma huella de confianza; solo la energía de entrada los separa. |
| Filtro de compresión | Un segmento con razón de compresión mayor que 2,4 solo se descarta si además parece no ser voz | En mlx esa razón se calcula por ventana de 30 s, no por segmento: descartar por ella sola borraba dictado real. Los bucles se limpian con el colapso de repeticiones. |
| Parar el micrófono | En un hilo con tope de 1,5 s | Parar un micrófono real puede bloquear a PortAudio y CoreAudio entre sí (mutex cruzado). Con audio simulado nunca pasa. Con el tope, el texto sale igual y el proceso termina con `os._exit`. Verificado con un micrófono real: cierra en unos 4 s. |
| Caché de MLX | Se libera solo al pasar 1 GB | Un archivo de 60 min llegaba a 3,05 GB de pico; liberando al pasar el límite, 1,2 GB. Liberar en cada grupo bajaba a 1,35 GB pero tardaba 5 veces más (42,9 min contra 7,6 min). |
| Reintento de descarga | Solo ante «no está en caché» (`LocalEntryNotFoundError` en la cadena de causas) o un modelo incompleto (config sin pesos) | Antes atrapaba cualquier error y mentía sobre la causa. La descarga corre en el hilo del modelo y la interfaz lee su avance, así no hay texto suelto que choque con el redibujado. |
| stdout | Solo la transcripción; la interfaz y los avisos a stderr | Que `voz \| pbcopy` y las redirecciones salgan limpios. |
| Interfaz | Solo con stdin y stderr en una terminal | Pipes y scripts reciben el flujo simple. `VOZ_UI=simple` es la salida de emergencia. |
| Sin dependencias de UI | ANSI y `termios` | Una dependencia menos que mantener y que auditar. |
| Glosario | Solo nombres propios, mínimo 3 letras, una pasada, valores literales | Las palabras comunes dan falsos positivos. La pasada única evita que una regla dispare otra, y los valores literales dejan escribir rutas como `C:\dev`. |

## Medidas de referencia

| Qué | Valor |
|---|---|
| Latencia de un tramo con el modelo caliente | 0,86 s en promedio (0,74 a 1,07 s). Con `auto`, unos 0,9 s más. |
| Carga inicial del modelo | Unos 4,75 s. |
| Primer texto provisional | 2,4 s después de empezar a hablar (frase de 8 s sin pausas). |
| Archivo de 54 s en frío | 7,9 s (unas 7 veces tiempo real), unos 790 MB de RAM. Modelo en disco: 1,5 GB. |
| Archivos largos | 30 min en 3,9 min; 60 min en 7,6 min con 1,2 GB de pico. |

| Qué | Valor |
|---|---|
| Motor en vivo, 30 min a velocidad doble | 0 pausas de más de 1 s en la interfaz, 4.604 palabras, RAM estable en 494 MB. |
| WER con voz sintética | Cerca de 12,6 % (sin contar números ni «gracias»). |
| Aciertos del detector de pausas según el ruido (casos con el número correcto de tramos) | Limpio 3/3 · 30 dB 4/9 · 20 dB 6/9 · 10 dB 1/9 · 5 dB 0/9, con unos 110 µs de CPU por cuadro de 30 ms. Es el límite honesto: por debajo de unos 10 dB haría falta un detector entrenado. |

# 7. Configuración

## 7.1 Archivos y carpetas

| Ruta | Qué es | Sobrescribible con |
|---|---|---|
| `~/.config/voz/config.toml` | Preferencias: modo de copia e idioma principal. | `VOZ_CONFIG` |
| `~/.config/voz/corrections.toml` | Glosario de correcciones. | (ninguna) |
| `~/.local/state/voz/` | Grabaciones a salvo: `sesion-AAAAMMDD-HHMMSS-gN.wav`, `.txt` y `.lock`. Se conservan las últimas 5 ejecuciones. Solo tu usuario puede leerlas (carpeta `0700`, archivos `0600`). | `VOZ_STATE_DIR` |
| `~/.cache/huggingface/` | Modelos descargados. | variables de Hugging Face |

Una hora de audio ocupa unos 115 MB. En el `.txt` de cada grabación se anotan las marcas `[descartada]` y `[restaurada]` cuando usas `⌫` y `u`.

## 7.2 `config.toml`

```
[portapapeles]
copiar = "auto"        # auto (default) | preguntar | nunca
```

- `auto`: cada grabación se copia al terminar.
- `preguntar` y `nunca`: no se copia sola y en la revisión aparece la tecla `s`. Hoy la interfaz los trata igual. Salir con «dejar la última» o «copiar todas» desde la pantalla de elegir sí copia.
- `--no-copy` gana siempre sobre este valor (y quita la tecla `s`). Un valor desconocido se trata como `auto`; un archivo ilegible avisa y usa los valores por defecto.

```
[idioma]
principal = "es"       # es | en
```

- Idioma al que se inclina `--lang auto` (primer tramo, volver a él con menos evidencia, parciales antes del primer tramo). Orden: este valor; si falta, `LC_ALL` / `LANG` solo si dicen `es_*` (un `en_US` heredado de la terminal no prueba nada); luego `defaults read -g AppleLocale` con un plazo corto: una región hispanohablante (AR BO CL CO CR CU DO EC ES GT GQ HN MX NI PA PE PR PY SV UY VE; EE. UU. no) da español aunque el locale sea `en_CO`, y si no, el idioma del propio locale si es `es` o `en`; después el `en` de `LANG` si lo hubo; si no, español. Un valor inválido avisa y pasa al idioma del sistema.

## 7.3 `corrections.toml`

```
[replacements]
"supa base" = "Supabase"      # clave: lo que Whisper escribe · valor: lo que querías
"cloud code" = "Claude Code"
```

- Las claves no distinguen mayúsculas, pero sí tildes (hoy no se pliegan).
- Las claves se ordenan de la más larga a la más corta: gana la regla de varias palabras.

- Claves de menos de 3 caracteres se ignoran con un aviso. Una regla que solo cambia mayúsculas (`"nit" = "NIT"`) es válida: normaliza cualquier variante de la palabra.
- En el modo en vivo se aplica por tramo; una regla de varias palabras partida por una pausa no se aplica. En archivos, por segmento.
- Reglas: solo nombres propios y términos técnicos. Nunca palabras comunes ni claves que existan en otro idioma (un `cloud` a `Claude` dañaría una frase en inglés).

### 7.4 Variables de entorno

| Variable | Quién la fija | Efecto |
|---|---|---|
| `HF_HUB_OFFLINE=1` | `bin/voz` | El modelo se resuelve solo desde la caché local. Si falta o está incompleto, `stt.download_model` pasa a `0` solo mientras descarga el snapshot (con el progreso en la interfaz), vuelve a `1` y sigue. |
| `HF_HUB_DISABLE_PROGRESS_BARS=1` | `bin/voz` | Sin barras de descarga de Hugging Face; el progreso lo dibuja la interfaz de `voz`. |
| `HF_HUB_DISABLE_TELEMETRY=1` | `bin/voz` | Sin telemetría de Hugging Face. |
| `TOKENIZERS_PARALLELISM=false` | `bin/voz` | Silencia un aviso de los tokenizadores. |
| `VOZ_UI=simple` | tú | Fuerza el flujo simple en vez de la interfaz interactiva. |
| `NO_COLOR` | tú | Quita los colores; el texto provisional se marca con ‹ ›. |
| `VOZ_CONFIG` | tú | Otra ruta para `config.toml`. |
| `VOZ_STATE_DIR` | tú | Otra carpeta para las grabaciones a salvo (útil en pruebas). |
| `VOZ_DIAG=1` | tú | Diagnóstico en vivo: una línea por tramo (sin texto) en `diag.log` de la carpeta de estado. Sirve para calibrar la puerta de silencio. |

### 7.5 Recuperar una sesión

Si `voz` murió sin cerrar bien, el siguiente arranque en modo micrófono imprime algo como la ruta del `.wav`, la del `.txt` y el comando `voz "<ruta>.wav"` para re-transcribirlo completo. Se avisa una sola vez por sesión.

### 7.6 Recuperar el texto con `voz copiar`

Si cerraste la terminal o copiaste otra cosa encima, `voz copiar --lista` muestra las grabaciones guardadas en las últimas 5 sesiones (fecha, duración, palabras y el comienzo de cada una), y `voz copiar N` copia la número N. Salta las descartadas y no modifica ningún archivo.

## 8. Tests y benchmarks

### 8.1 Tests

```
./.venv/bin/python -m pip install -r requirements.lock.txt -r requirements-dev.txt
./.venv/bin/python -m pytest -q                  # unitarios: 229 tests, sin modelo ni micrófono
./.venv/bin/python -m pytest -q -m integration   # 3 E2E con el modelo real en caché
```

`pyproject.toml` excluye por defecto el marcador `integration`. Los de integración usan `say` para generar voz y se saltan solos si el modelo no está en caché.

| Archivo | Qué cubre |
|---|---|
| `test_cli.py` | El parser y sus validaciones. |
| `test_flow.py` | Cada transición de la máquina de estados. |

| Archivo | Qué cubre |
|---|---|
| `test_live.py` | `Segmenter` y `LiveSession`. |
| `test_record.py` | Recorte de silencio y umbrales. |
| `test_stt_clean.py` | Filtros de alucinación y colapso de repeticiones. |
| `test_corrections.py` | Carga y aplicación del glosario. |
| `test_output.py`, `test_autosave.py` | Subtítulos, JSON, no sobrescribir; escritura a disco, bloqueos y poda. |
| `test_ui.py` | Teclas, medidor, ajuste de texto, modos de copia. |
| `test_concurrency.py` | Regresiones de concurrencia: `drain`, calentamiento, escritor de autosave, parada del micrófono. |
| `test_keys_clipboard.py` | Teclas, portapapeles y códigos de salida del Shell interactivo, y los avisos mientras carga el modelo. |
| `test_stt_glossary.py` | Limpieza de STT (alucinaciones, repeticiones), glosario y manejo de datos. |
| `test_regressions.py` | Regresiones de los fallos principales: carga del modelo, tramos que fallan, archivos y códigos de salida. |
| `test_config.py` | Configuración robusta: tipos incorrectos y archivos que no son UTF-8; idioma principal (config, `LANG`, región de `AppleLocale`, español). |
| `test_model_download.py` | Estado del modelo (ausente, incompleto, disponible), descarga con progreso simulado y cómo la dibuja la interfaz. |
| `test_shell_states.py` | Terminal estrecha (40, 59 y 80 columnas), micrófono denegado o mudo, textos de revisión y de `Ctrl-C`, transcripción incompleta. |
| `test_doctor.py` | `voz doctor`: advertencias frente a problemas críticos, estado de los modelos y símbolos. |
| `test_sticky_lang.py` | Idioma pegajoso (`LangState`): sesgo al principal, fronteras exactas, recuperación de un cambio falso, y que errores, tramos filtrados y parciales no muevan el idioma. |
| `test_integration.py` | E2E con el modelo real (marcador `integration`). |

Cuando arregles un bug, escribe primero el test que falla. Es la regla del proyecto (ver `CONTRIBUTING.md`).

## 8.2 Benchmarks

La carpeta `bench/` tiene scripts, pero **no incluye audios**: son grabaciones tuyas o sintéticas que debes generar. Todos dependen del modelo o de `say` y `ffmpeg` de macOS.

| Script | Para qué | Necesita |
|---|---|---|
| `bench/bench_src.py` | Banco de ruido del detector de pausas: mezcla tus 3 frases con ruido blanco, rosa y de calle a 30, 20, 10 y 5 dB, y cuenta cuántas veces el `Segmenter` real acierta el número de tramos. | `bench/audio/f1.wav`, `f2.wav`, `f3.wav` |
| `bench/long/make_long.py` | Genera audios largos (1, 3 y 5 min; con argumentos, 10, 30 y 60) mezclando órdenes en español, dictado técnico e inglés. | `say`, `ffmpeg` |
| `bench/long/stress_live.py` | Alimenta el motor en vivo (modelo real) con un audio largo a ritmo real mientras mide el retraso de una interfaz simulada. | modelo en caché |
| `bench/long/pty_ui.py` | Maneja la interfaz real en una pseudo-terminal y detecta congelamientos (el cronómetro deja de avanzar). | `voz` instalado en `~/.local/bin` |

| Script | Para qué | Necesita |
|---|---|---|
| `bench/ux/pty_flows.py` | Verificaciones de extremo a extremo de los flujos de teclas con audio simulado y un `pbcopy` falso. | `voz` instalado y `bench/long/long1_pausas.wav` |

### 8.3 Grabar tus 3 frases para `bench_src.py`

El script carga `audio/f1.wav`, `f2.wav` y `f3.wav` desde `bench/audio/` y se corre desde `bench/`. Cada frase debe ser una oración de unos 5 a 10 segundos con una o dos pausas largas (más de 0,7 s) en medio, en WAV de 16 kHz mono.

Las etiquetas de «dónde hay voz» se calculan con un umbral absoluto de -45 dBFS, pensado para un silencio digital limpio. Por eso conviene **voz sintética** (el silencio de `say` es silencio digital). Una grabación real con ruido de sala por encima de ese nivel se marcaría toda como voz y el banco no tendría sentido.

```
mkdir -p bench/audio
say -v Paulina -o "$TMPDIR/f1.aiff" "Necesito dos galones de salsa rosada. [[slnc 1200]] El pago es
mañana."
say -v Paulina -o "$TMPDIR/f2.aiff" "Revisa que el pedido quede confirmado. [[slnc 1500]] Luego avísame por
voz."
say -v Paulina -o "$TMPDIR/f3.aiff" "Probemos el detector con ruido de fondo. [[slnc 1000]] Y ahora la
segunda parte."
for n in 1 2 3; do ffmpeg -y -i "$TMPDIR/f$n.aiff" -ar 16000 -ac 1 bench/audio/f$n.wav; done

cd bench && ../.venv/bin/python bench_src.py
```

(`[[slnc N]]` inserta N milisegundos de silencio; las pausas naturales de `say` son de unos 250 ms, menos que el corte de 700 ms.) La salida es una tabla con, para cada nivel de ruido, los aciertos y cuánto del inicio de la frase se pierde, más el costo de CPU en microsegundos por cuadro. Es la forma de comprobar que un cambio al detector no empeora el ruido.

## 9. Guía para forks

### 9.1 Enchufar otro motor de transcripción

El motor vive en `stt.py`. El resto del programa depende de este contrato, no de MLX:

```
transcribe_segments(audio, *, model, language, initial_prompt=None,
                    condition_on_previous_text=False, dictation=True,
                    lang_state=None)       # LangState: idioma pegajoso en vivo con language="auto"
    -> list[{"start": float, "end": float | None, "text": str}]
transcribe(audio, **kw) -> str            # los segmentos unidos
stt.last_language                         # idioma de la última llamada, para la interfaz
detect_language_probs(audio, model) -> {"es": p, "en": p}   # LangState.choose decide
```

`audio` puede ser una ruta de archivo o un arreglo float32 mono a 16 kHz. `voz.py` inyecta la función `transcribe` en `LiveSession` y en `liveui.run`, así que cambiar de motor es reemplazar esas funciones.

1. Implementa `transcribe_segments` con tu motor y devuelve segmentos con el formato de arriba.
2. Reutiliza los filtros: `_filter_segments` espera un diccionario `{"segments": [...]}` donde cada segmento trae `text`, `start`, `end`, `no_speech_prob`, `avg_logprob` y `compression_ratio`. faster-whisper expone esos mismos campos en sus segmentos (por verificar contra la versión que uses); adáptalos a diccionarios y no pierdes la defensa contra alucinaciones.
3. Implementa `detect_language_probs` y `detect_languages` para `--lang auto` (`LangState` solo consume las probabilidades).
4. Revisa los puntos que hoy tocan MLX directamente: en `stt.py`, `transcribe_segments` (`mlx_whisper.transcribe`), `detect_language_probs`, `detect_languages` y la liberación de caché (`mx.clear_cache`) en `transcribe_file_segments`; en `liveui.py`, `Audio._open_sim` (`mlx_whisper.audio.load_audio`); en `doctor.py`, el chequeo del import y de la caché de modelos.

5. Ajusta `DEFAULT_MODEL` y `ACCURATE_MODEL` a los nombres de tu motor, y las pruebas de `test_stt_clean.py` que fijan el formato de resultado.

## 9.2 Portar a Windows o Linux

| Pieza | Dónde | Qué cambiar |
|---|---|---|
| Portapapeles | `output.py` (`to_clipboard`, `from_clipboard`) | Son dos funciones. `wl-copy` o `xclip` en Linux; `clip` o PowerShell en Windows. Cuida que reciban UTF-8. |
| Captura de audio | `record.py` (`open_input`) | `sounddevice` ya es multiplataforma, así que debería bastar. La parada con tope de 1,5 s es una defensa contra un fallo de CoreAudio: en otras plataformas es inofensiva. |
| Permisos de micrófono | `voz.py` (`_warn_silence`), `doctor.py` | El aviso habla de Ajustes del Sistema y de `tccutil`. Cámbialo por el equivalente de cada sistema. |
| Teclado de la terminal | `ui.py` (`Keys`) | Usa `termios`, `tty` y `select` sobre el descriptor de stdin: nada de eso existe en Windows. Ahí harían falta `msvcrt` o una biblioteca de entrada. |
| Bloqueos de archivo | `autosave.py` (`fcntl.flock`) | Solo POSIX. En Windows, `msvcrt.locking` u otro mecanismo para detectar sesiones huérfanas. |
| Rutas de estado y de configuración | `autosave.py`, `config.py`, `corrections.py` | Siguen la convención de Linux (`~/.config`, `~/.local/state`); en Windows conviene usar `%APPDATA%` y `%LOCALAPPDATA%`. |
| Señales | `voz.py` (`_enable_stack_dump`) | `SIGQUIT` no existe en Windows; el código ya lo tolera y simplemente no registra el volcado. |
| Lanzador | `bin/voz` | Es un script `sh`. Para Windows, un `.cmd` o un punto de entrada de Python. |
| Pruebas | `tests/test_integration.py` | Usan `say` (solo macOS). Sustitúyelo por otro sintetizador o por audios de prueba. |

> Recomendación: un PR por pieza (portapapeles, luego teclado, luego motor) y el camino de macOS siempre con sus tests en verde. Abre un issue antes para acordar la interfaz.

## 9.3 Añadir un subcomando

Los subcomandos son valores del argumento posicional `target`. `voz.py` los lista en la tupla `COMMANDS` para no confundirlos con un archivo. Pasos:

1. Añade el nombre a `COMMANDS`.
2. En `main`, antes de cargar el glosario, añade una rama como la de `doctor`.
3. Implementa el comando en un módulo nuevo de `src/` con imports perezosos, para que `--help` siga siendo instantáneo.
4. Si necesita banderas nuevas, agrégalas al parser y valida las combinaciones en `parse_args`, como las demás.
5. Añade pruebas del parser en `tests/test_cli.py` y del módulo en su propio archivo.

```
# src/voz.py
COMMANDS = ("devices", "doctor", "ejemplo")

def main(argv=None):
    ...
    if args.target == "ejemplo":
        import ejemplo            # src/ejemplo.py, con run() -> int
        return ejemplo.run()
```

Una futura `voz habla` (texto a voz) encajaría exactamente así.

### 9.4 Cómo contribuir y versionar

El proyecto usa GitHub Flow. El detalle y la plantilla de PR están en `CONTRIBUTING.md` y `.github/`; este es el resumen.

- **Ramas.** `main` está siempre estable y protegida. Todo cambio entra por un pull request desde una rama corta: `feat/…`, `fix/…`, `docs/…` o `chore/…`. El PR se integra con squash.
- **Commits.** Conventional Commits: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`.
- **Versiones.** Semánticas, con tags `vX.Y.Z` y un release en GitHub cuyas notas salen del `CHANGELOG.md`. El changelog sigue Keep a Changelog: lo nuevo va en *Unreleased* y esa sección se cierra al publicar.
- **Tests.** Todo cambio de comportamiento lleva su test; los unitarios corren con `pytest -q` y no necesitan modelo ni micrófono.
- **Entrada.** Los issues con la etiqueta `good first issue` son el mejor punto de partida.

> No hay ramas `develop` ni `release`: es un proyecto pequeño con un solo mantenedor y GitHub Flow basta.

## 10. Limitaciones conocidas y roadmap

### 10.1 Limitaciones

| Limitación | Detalle |
|---|---|
| Solo macOS con Apple Silicon | MLX, `pbcopy` / `pbpaste` y CoreAudio. |
| Deletreo pobre | Siglas, letras sueltas, rutas y claves salen como Whisper las escriba. Esta versión no las borra, pero tampoco las reconoce mejor. |
| Sin diarización | No distingue quién habla. Todo es un solo texto. |
| Glosario sensible a tildes | `nequi` y `néqui` son claves distintas; no hay normalización Unicode. |
| Reglas de varias palabras en vivo | No se aplican si una pausa parte la frase en dos tramos. |
| Idiomas | En `auto` solo español e inglés. La sesión mantiene su idioma y solo cambia con evidencia fuerte; en llamadas o con varias voces conviene `--lang es`. Mezclar idiomas dentro de una frase sin pausa enreda a Whisper. Y `--lang` no se valida: un código inválido falla más tarde con un mensaje poco claro. |
| Ruido fuerte | Con 10 dB de relación señal-ruido o peor el detector de pausas falla. |
| Ruido blanco puro | Puede producir una palabra alucinada. |
| «gracias» aislado | Con el micrófono se conserva si hubo voz real (solo se filtra si el audio es silencio); en archivos, aislado, se filtra por prudencia. |
| Solo micrófono | No captura el audio del sistema (videollamadas). Usa el dispositivo de entrada por defecto de macOS; para otro, cámbialo en los ajustes de sonido del sistema. |

### 10.2 Roadmap

Sin fechas. Es un proyecto personal que avanza a ratos.

1. **v0.1.1.** Glosario insensible a tildes (NFC y plegado de vocales), portapapeles con entorno UTF-8, validar `--lang`, señal sonora al terminar, tecla `e` para corregir una palabra y recordarla, tecla `i` para cambiar el idioma y reprocesar, idioma principal aprendido del uso, pista de vocabulario opcional y CI en GitHub Actions. Lista priorizada en `CHANGELOG.md`.
2. **Windows y Linux** con un motor alternativo (faster-whisper u otro) detrás del contrato de la sección 9.1, y portapapeles y teclado multiplataforma.

3. **`voz habla`**, a futuro: texto a voz, el camino inverso.

Ideas en estudio, que piden medir o instalar algo antes de decidir: administrar el glosario desde la terminal, un modo de deletreo opcional (primero hace falta un corpus real de cómo escribe Whisper las letras), elegir el dispositivo de entrada, perfiles de ruido medidos, captura del audio del sistema con un dispositivo virtual y diarización solo para archivos. Grabar a 48 kHz y el `initial_prompt` están descartados (sección 6).

## 11. Licencia

`voz` se publica bajo la **licencia MIT**. Copyright (c) 2026 Daniel Patiño Vargas. El texto completo está en el archivo `LICENSE` del repositorio. Las dependencias (mlx-whisper, MLX, sounddevice, NumPy, SciPy, torch y los pesos de los modelos de Whisper) tienen cada una su propia licencia; revísalas si redistribuyes algo.

Lo construimos Daniel y Claude (IA de Anthropic) en colaboración.